Class Inheritance

A recap of Car class with a slightly different version:

- Use an arrow function for the method.
    - The advantage here is that it makes the syntax more concise.
    - Because the return value is implicit and there's only one line of logic, we can omit the curly braces and return keyword:
- Have a default class property:
    - A class can have properties which are not set by parameters. In this case they have default values assigned in the constructor
    - Give Car an additional property, 'onRoad', which is a boolean, with a default value of true.

```
    class Car {
        constructor(make, model, year, color) {
            this.make = make;
            this.model = model;
            this.year = year;
            this.color = color;
            this.onRoad = true;
            this.brag = () => `Check out my awesome ${this.year}
${this.make} ${this.model}!`;
        }
    }
```

A method of the class does not have to be defined in the constructor. We could have it separate from the constructor.

Differences with method not in the constructor: - this keyword does not appear before the name of the method - nor does the function keyword come before the method name

```
class Car {
  constructor(make, model, year, color) {
    this.make = make;
    this.model = model;
    this.year = year;
    this.color = color;
    this.onRoad = true;
  }
  brag() {
    return `Check out my awesome ${this.year} ${this.make}
${this.model}!`;
  }
}
```

Instantiate a car object and log a few properties, including the brag method:

```
let myCar = new Car('Porsche', '356C', 1964, 'red');
console.log(myCar.make);
console.log(myCar.model);
console.log(myCar.onRoad);
console.log(myCar.brag());
// Check out my awesome 1964 Porsche 356C!
```

Class Inheritance:

A Class can be declared that inherits all the methods and properties of another, parent, class. The **extends** keyword is used to express this relationship. In addition to the constructor method, the child class has a **super()** method that takes all the parameters unique to the new parent class.

Let's start with a class, `Player`, with two properties, `name` and `energy`. Only `name` has a parameter to set. The `energy` property haw a default value of 100, so no parameter to set.

```
class Player {

    constructor(name) {
        this.name = name;
        this.energy = 100;
    }
}
```

Give the class two methods: `introducePlayer`, which returns a string and `restPlayer` that sets the energy level and announces the update.

```
class Player {

    constructor(name) {
        this.name = name;
        this.energy = 100;
    }

    introducePlayer() {
        return `Player name: ${this.name}\nGame on!`;
    }

    restPlayer() {
        let energy = this.energy;
        this.energy += 10;
        return `Player rested..\nEnergy increased from ${energy} to
${this.energy}.`;
    }

}
```

Instantiate an object of the `Player` class and test out its properties and methods:

```
let player = new Player('Wezzul');

console.log(player.name);
console.log(player.energy);
console.log(player.introducePlayer());
console.log(player.restPlayer());
```

extending an existing class

Now for the inheritance part. We'll make a new class called `Warrior` which extends Player. This means that `Warrior` will start off with all the properites and methods of `Player`. On top of that, `Warrior` will have its own unique properites and methods.

Declare a new class with the syntax `class Warrior extends Player`:

```
class Warrior extends Player {
}
```

The constructor has two parameters: - `name` : the inherited parameter from the Player - `weapon` : unique parameter of the Warrior class

```
class Warrior extends Player {
    constructor(name, weapon) {
    }
}
```

super() Classes that extend have an additional method besides the constructor, called `super()`. - `super()` goes inside the constructor. - `super()` has as its parameters all inherited properties that require arguments to be passed in.

```
class Warrior extends Player {

constructor(name, weapon) {
    super(name);
}

}
```

After `super()` method come the properties unique to the Warrior class. We'll keep it short and only have one propert: `weapon`:

```
    class Warrior extends Player {

    constructor(name, weapon) {
        super(name);
        this.weapon = weapon;
    }

    }
```

Give the Warrior class its own method, `attack()`. Based on the energy level, it returns a message announcing an attack, or it says that it doesn't have enough energy to attack.

Each time the `attack()` method is called, the energy level decreases by 10. This is followed by an announcement of the lower energy level.

```
class Warrior extends Player {

  constructor(name, weapon) {
    super(name);
    this.weapon = weapon;
  }

  attack() {

    let msg = "";
    if (this.energy <= 0) {
      msg = `Not enough energy to wield the ${this.weapon}!`;
    } else {
      msg = `Warrior attacked with the ${this.weapon}!`;
    }

    let energy = this.energy;
    this.energy -= 10;
    msg += `\nEnergy decreased from ${energy} to ${this.energy}.`;
    return msg;
  }

}
```

Instantiate an instance of the Warrior class, passing in two arguments: name and weapon.

```
    const warrior = new Warrior("Konan", "Battle Ax");

    console.log(warrior.name);
    console.log(warrior.energy);
    console.log(warrior.weapon);
    console.log(warrior.introducePlayer());
```

```
    console.log(warrior.restPlayer());
    console.log(warrior.attack());
```

ineriting inheritance

A class can extend a class that extends a class. Let's make another class, `SuperHero` that extends `Warrior`.

`SuperHero` will start out with everything the `Warrior` class has, which includes everything it inherited from the `Player` class.

We will give the `SuperHero` class its own property, `superPower`, as well as its own method `useSuperPower()`.

The `useSuperPower()` method is virtually identical to the `attack()` method, so you can copy-paste it:

```
- If there is enough energy to use the superpower, the superpower is used
and that fact is announced
- Else, it says that it doesn't have enough energy to use the super power.
- Also, as with `attack()`, calling the `useSuperPower()` method costs 10
units of energy.
```

The `super()` method now has two parameters, `name` and `weapon`, which it inherits from the `Player` and `Warrior` classes, respectively.

```
    class SuperHero extends Warrior {

        constructor(name, weapon, superPower) {
            super(name, weapon);
            this.superPower = superPower;
        }

        useSuperPower() {

            let msg = "";
            if (this.energy <= 0) {
            msg = `Not enough energy to use ${this.superPower}!`;
            } else {
            msg = `Super Hero used ${this.superPower}!`;
            }

            let energy = this.energy;
            this.energy -= 10;
            msg += `\nEnergy decreased from ${energy} to ${this.energy}.`;
            return msg;
        }

    }
```

Instantiate an instance of the `SuperHero` class, passing in the three arguments: `name`, `weapon` and `superPower`:

```
    const superhero = new SuperHero("Magami", "Battle Ax", "Super-
Strength");

    console.log(superhero.introducePlayer());
    console.log(superhero.attack());
    console.log(superhero.restPlayer());
    console.log(superhero.superPower);
    console.log(superhero.useSuperPower());
```

Prototype

The **prototype** keyword is used to add properties to a class.

Let's add a boolean property `wearsCape` to the `SuperHero` class.

```
    SuperHero.prototype.wearsCape = true;
```

Also add a method called `introduceSuperHero`:

```
    SuperHero.prototype.introduceSuperHero = function() {
    return `Behold! I am ${this.name}! I wield a fearsome ${this.weapon},
and I possess the incredible superpower of ${this.superPower}!`;
    }
```

Instantiate a new `SuperHero` object, which will have the new property `wearsCape` as well as the new method, `introduceSuperHero`.

```
    const superhero3 = new SuperHero("Zogg", "Laser Gun", "X-Ray Vision");

    // Log the whole object to see if energy and hasCape are in
    console.log(superhero3);

    console.log(superhero3.introducePlayer());
    console.log(superhero3.attack());
    console.log(superhero3.restPlayer());
    console.log(superhero3.useSuperPower());
    console.log(superhero3.introduceSuperHero());
    console.log(superhero3.wearsCape);
```

OOP CLASS LAB EXERCISES:

Complete these classes, Mammal and HouseCat, by filling in the missing data. You need to look at all the code to get clues.

See the output of the desribe method to know how to make it.

```
class Mammal {

constructor(name, legs, ???, ???) {
    this.name = name;
    // ???
    this.continent = continent;
    // ???
}

describe() {
    // ???
}

}

let animal1 = new Mammal('???');
console.log(animal1.name); // zebra
console.log(animal1.legs); // 4
console.log(animal1.continent); // Africa
console.log(animal1.herbivore); // true
console.log(animal1.describe);
// The zebra is a 4-legged herbivorous mammal native to Africa.

let animal2 = new Mammal('???');
console.log(animal2.name); // tiger
console.log(animal2.legs); // 4
console.log(animal2.continent); // Asia
console.log(animal2.herbivore); // false
// The tiger is a 4-legged carnivous mammal native to Asia.

let animal3 = new Mammal('???');
console.log(animal3.name); // kangaroo
console.log(animal2.legs); // 2
console.log(animal2.continent); // Australia
console.log(animal2.herbivore); // true
// The kangaroo is a 2-legged herbivorous mammal native to Australia.

let animal4 = new Mammal('???');
console.log(animal4.name); // llama
console.log(animal4.legs); // 4
console.log(animal4.continent); // South America
console.log(animal4.herbivore); // true
// The llama is a 4-legged herbivorous mammal native to South America.
```

Make a class `HouseCat` that extends `Mammal`.

In addition to its inherited properties, it has 3 new ones: `age`, `toys` and `owner`. - `toys` is an array with at three toy items - `owner` is an object with properties of `name`, `age` and `job`.

Give the `HouseCat` class a method called meow, as shown. It takes a name as argument, concatenates a personalized greeting and returns it. See the outputted greeting for clues as to how to make it.

```
    class HouseCat extends Mammal {

        constructor(name, legs, ???, ???, ???, ???, ???) {

            super(name, legs, ???, ???);
            this.age = age;
            this.toys = toys;
            this.owner = owner;

        }

        sayMeow(name) {
            return `Meow, ${name}! My name is ${this.name}! I am a
${this.age}-year-old cat! My favorite toy is my ${this.toys[0]}! My
human's name is ${this.owner.name}!`;
        }

    }

    const toys = ['Floppy Fish', 'tennis ball', 'catnip'];
    const catOwner = {name: 'Sue', age: 42, job: 'teacher'};

    let cat = new HouseCat('Fluffy', 4, '???', '???', '???', '???',
'???');

    console.log(cat.legs); // 4
    console.log(cat.name); // Fluffy
    console.log(cat.herbivore); // true
    console.log(cat.continent); // North America
    console.log(cat.owner.name); // Sue
    console.log(cat.toys[1]); // tennis ball
    console.log(cat.toys[toys.length-1]); // catnip
    console.log(cat.job); // undefined
    console.log(cat.owner.job); // teacher

    console.log(cat.sayMeow('Bob'));
    // Meow, Bob! My name is Fluffy! I am a 3-year-old cat! My favorite
toy is my Floppy Fish! My human's name is Sue!
```

Using above as a template

Create a Magician class that has the following properties and methods

properties: name,health, magicPower

methods: sayName() getHealth() playerRest()

Create a subclass called Wizard that inherits from Magician and has the follow properties and methods

properties: magicStaff

methods: useMagic() maigcDrement() manaIncrement() checkMana()

OOP CLASS LAB EXERCISE SOLUTION:

Complete these classes by filling in the missing data. You need to look at all the code to get clues.

For the `describe` method, use if-else logic to produce the string `herbivorous` or `carnivorous`.

The comment output of the `describe` method tells us that the myster properties are `continent` and `herbivore`.

```
    class Mammal {

    constructor(name, legs, continent, herbivore) {
        this.name = name;
        this.legs = legs;
        this.continent = continent;
        this.herbivore = herbivore;
    }

    describe() {

        let diet = "";

        if(this.herbivore) {
            diet = 'herbivorous';
        } else {
            diet = 'carnivorous';
        }
        return `The ${name} is a ${legs}-legged ${diet} mammal native to
${continent}`;

    }

    }
```

Instantiate objects of the class.

```
    let animal1 = new Mammal('zebra', 4, 'Africa', true);
    console.log(animal1.name); // zebra
    console.log(animal1.legs); // 4
    console.log(animal1.continent); // Africa
    console.log(animal1.isHerbivore); // true

    let animal2 = new Mammal('tiger', 4. 'Asia', false);
```

```
    console.log(animal2.name); // tiger
    console.log(animal2.legs); // 4
    console.log(animal2.continent); // Asia
    console.log(animal2.herbivore); // false

    let animal3 = new Mammal('kangaroo', 2, 'Australia' true);
    console.log(animal3.name); // kangaroo
    console.log(animal2.legs); // 2
    console.log(animal2.continent); // Australia
    console.log(animal2.herbivore); // true

    let animal4 = new Mammal('llama', 4, 'South America', true);
    console.log(animal4.name); // llama
    console.log(animal4.legs); // 4
    console.log(animal4.continent); // South America
    console.log(animal4.herbivore); // true
```

Make a class `HouseCat` that extends `Mammal`. In addition to its inherited properties, it has 3 new ones: `age`, `toys` and `owner`. - `toys` is an array with at least two items as strings - `owner` is an object with properties of name, age and job

Give the HouseCat class a method called `sayMeow`, as shown. It takes a name as argument, so that the greeting can be personalized.

```
    class HouseCat extends Mammal {

        constructor(name, legs, continent herbivore, age, toys, owner) {

            super(name, legs, continent, herbivore);
            this.age = age;
            this.toys = toys;
            this.owner = owner;

        }

        sayMeow(name) {
            return `Meow, ${name}! My name is ${this.name}! I am a
${this.age}-year-old cat! My favorite toy is my ${this.toys[0]}! My
human's name is ${this.owner.name}!`;
        }

    }

    let cat = new HouseCat('Fluffy', 4, 'North America', false, 3,
['Floppy Fish', 'ball', 'catnip'], {name: 'Sue', age: 42, job:
'teacher'});

    console.log(cat.legs); // 4
    console.log(cat.name); // Fluffy
    console.log(cat.isHerbivore); // true
    console.log(cat.continent); // North America
```

```
    console.log(cat.owner.name); // Sue
    console.log(cat.toys[1]); // tennis ball
    console.log(cat.toys[toys.length-1]); // catnip
    console.log(cat.job); // undefined
    console.log(cat.owner.job); // teacher

    console.log(cat.sayMeow('Bob'));
    // `Meow, Bob! My name is Fluffy! I am a 3-year-old cat! My favorite
toy is my Floppy Fish! My human's name is Sue!
```